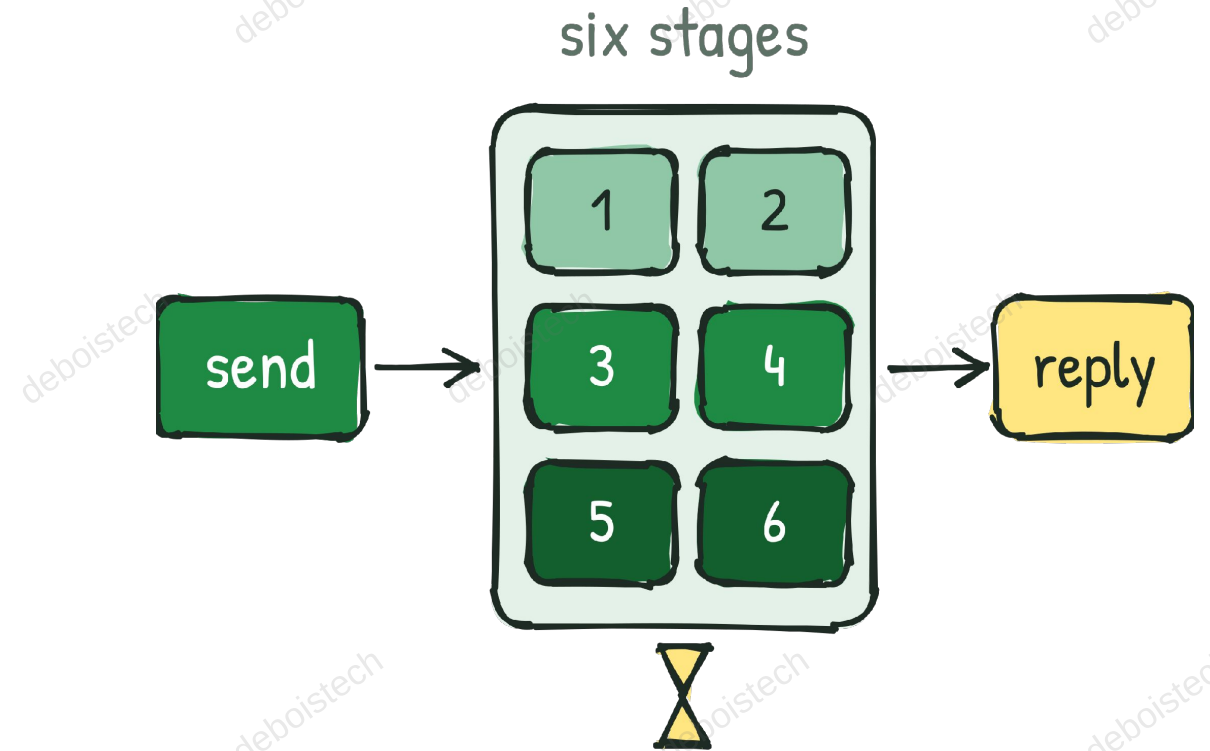


Inference Basics

What happens between "send" and "response"

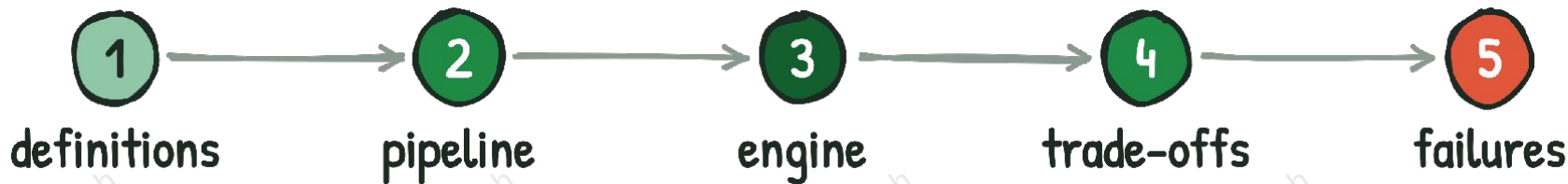


DAY 4 · ASTRA · ONLINE SESSION

The hook

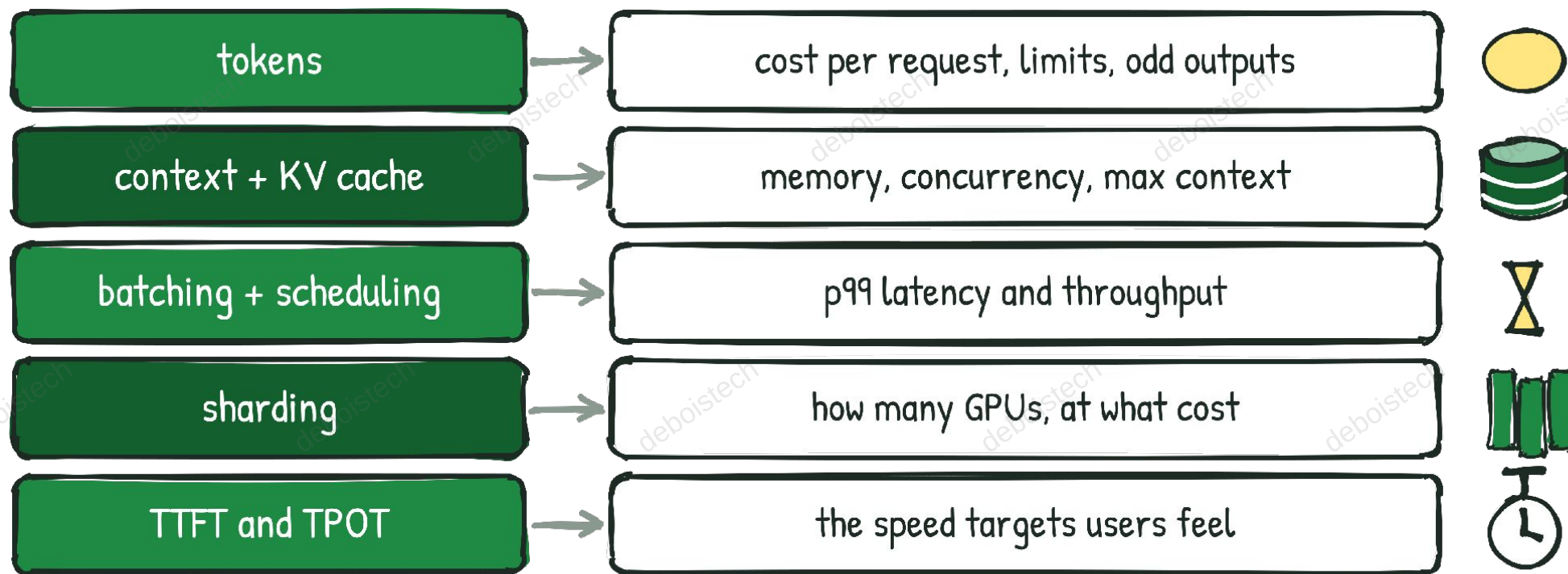
Diagram illustrating the question: "what do they actually measure?"

The diagram shows five yellow boxes at the top, each containing a metric: TTFT, TPOT, latency, throughput, and streaming. Arrows from each of these boxes point towards a central point below them, where a red question mark is located. Below the question mark is a larger white box with a black border containing the text "what do they actually measure?".



Why a production engineer cares

Calling a model is the easy part. In production you own what happens around it: speed, cost, capacity and reliability. Each idea today maps to a decision you will be asked to make. Which would hurt you first? Tell us in the chat.



chat: which of these would hurt you first?

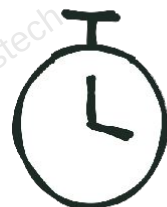
Quick recap from Day 1

A one-line refresher of the five words from Day 1. Before we move on, type the full form of TTFT in the chat. (Latency is how fast one request starts; throughput is how many we serve at once.)



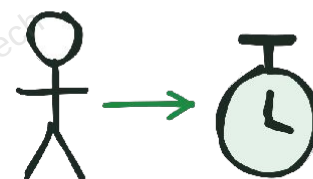
TTFT

time to first token



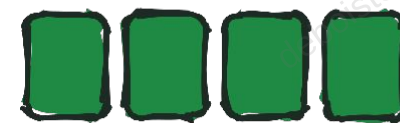
TPOT

time per output token



Latency

how fast it starts
+ throughput: how many



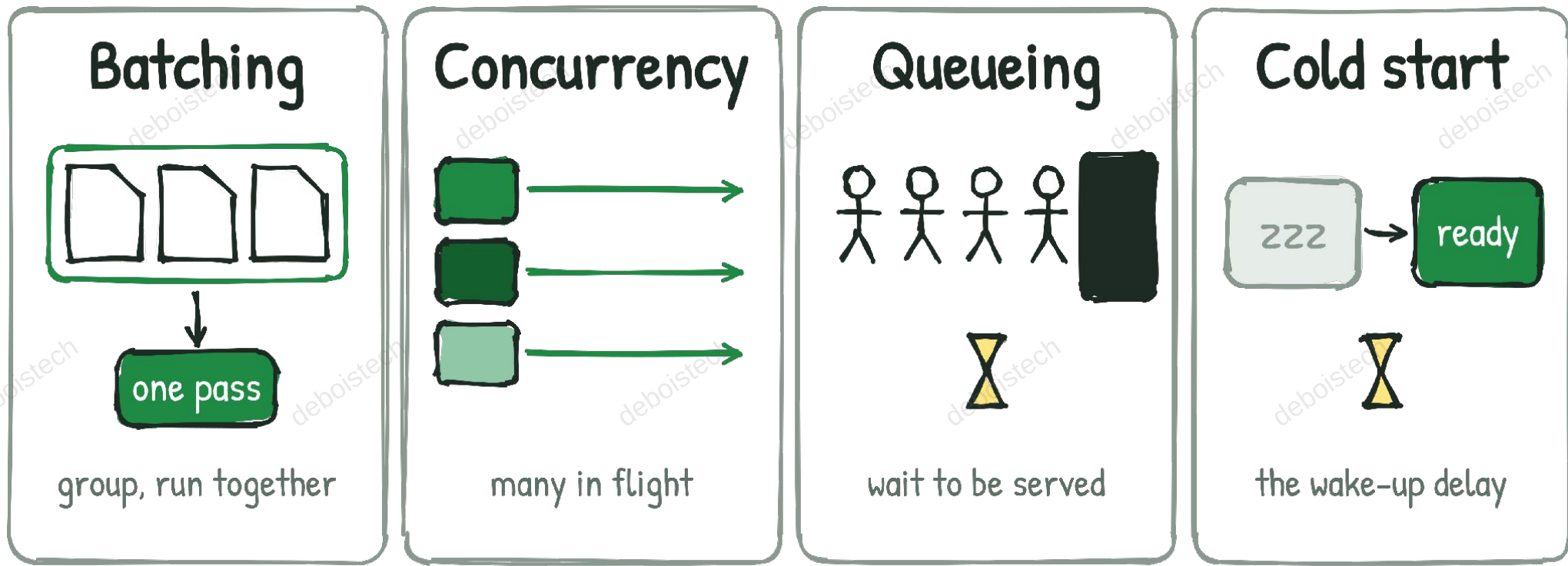
Streaming

tokens sent as made

chat: type the full form of TTFT

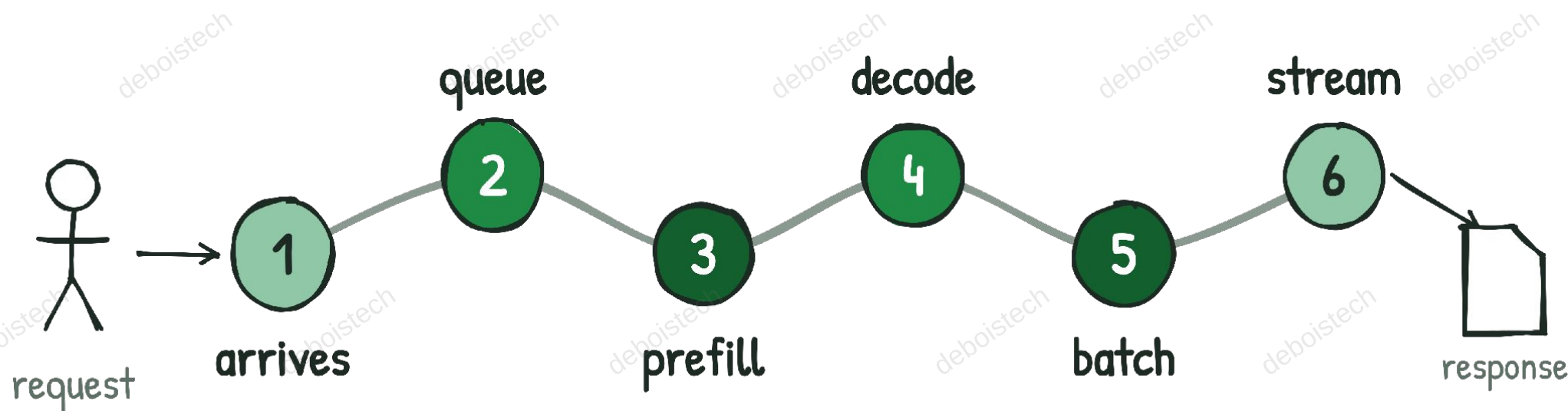
New terms for today

Four ideas decide real-world speed: batching (process requests together), concurrency (many in flight at once), queueing (waiting for a free slot) and cold starts (the delay before a sleeping model can answer).



A request's actual journey

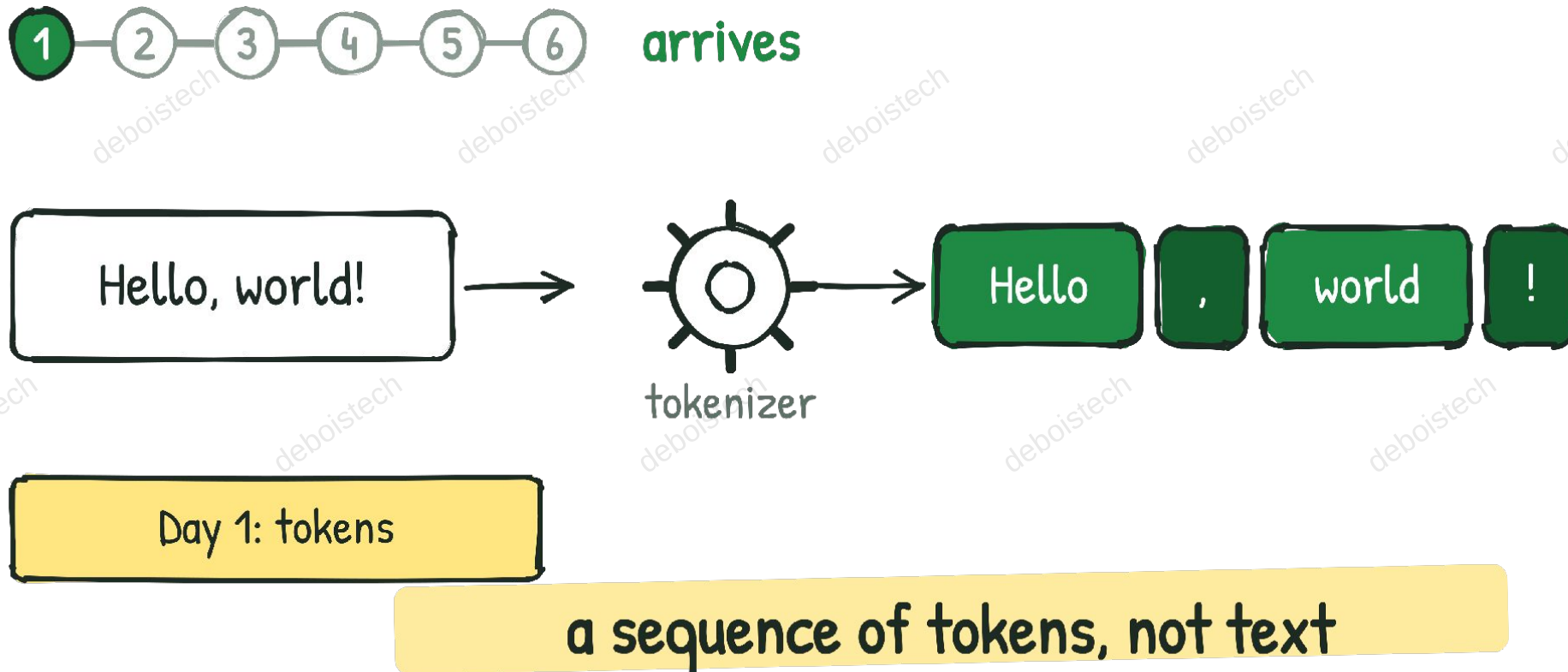
Every request passes the same six stages: it arrives, may wait in a queue, has its prompt processed (prefill), generates tokens one by one (decode), shares the GPU with others (batching), and streams back.



we will walk every stage

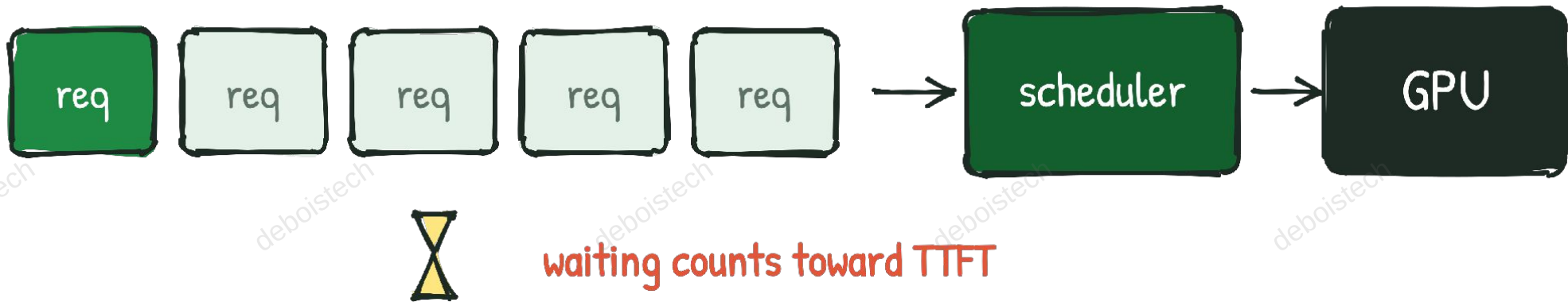
Stage 1: request arrives

The tokenizer from Day 1 cuts your text into tokens and the request is handed to the server. From here on, everything the model sees is a sequence of token IDs, not words.



Stage 2: queueing

The server may not start your request immediately. If every slot is busy, it waits in a queue. That waiting time is part of TTFT, and under heavy load it often dominates it.



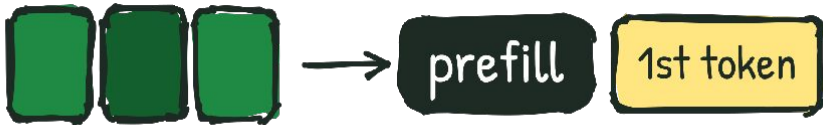
under load: a queue problem, not a model problem

Stage 3: prefill

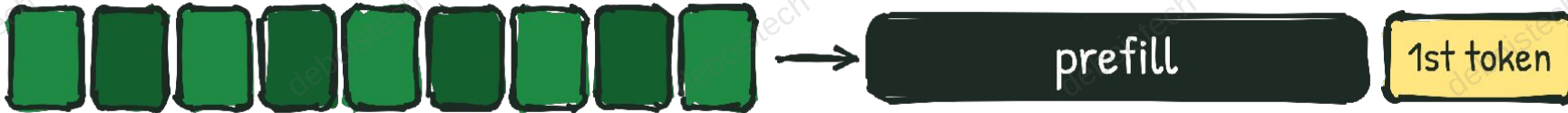
Prefill: the model reads the entire prompt in one pass, builds its internal state, then emits the first token. A longer prompt means more work before that first token, so a higher TTFT.



short prompt



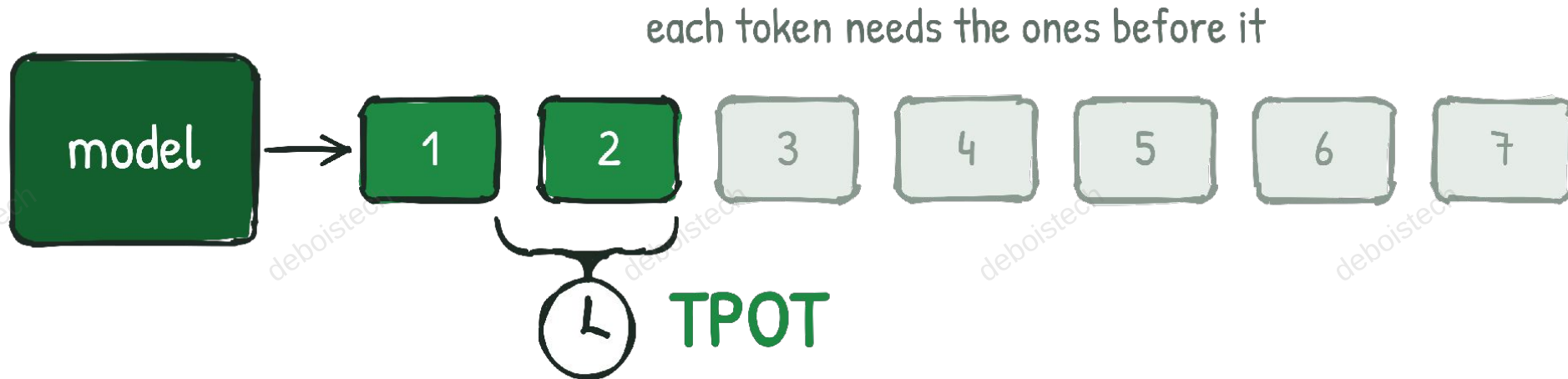
long prompt



longer prompt = longer prefill = higher TTFT

Stage 4: decode

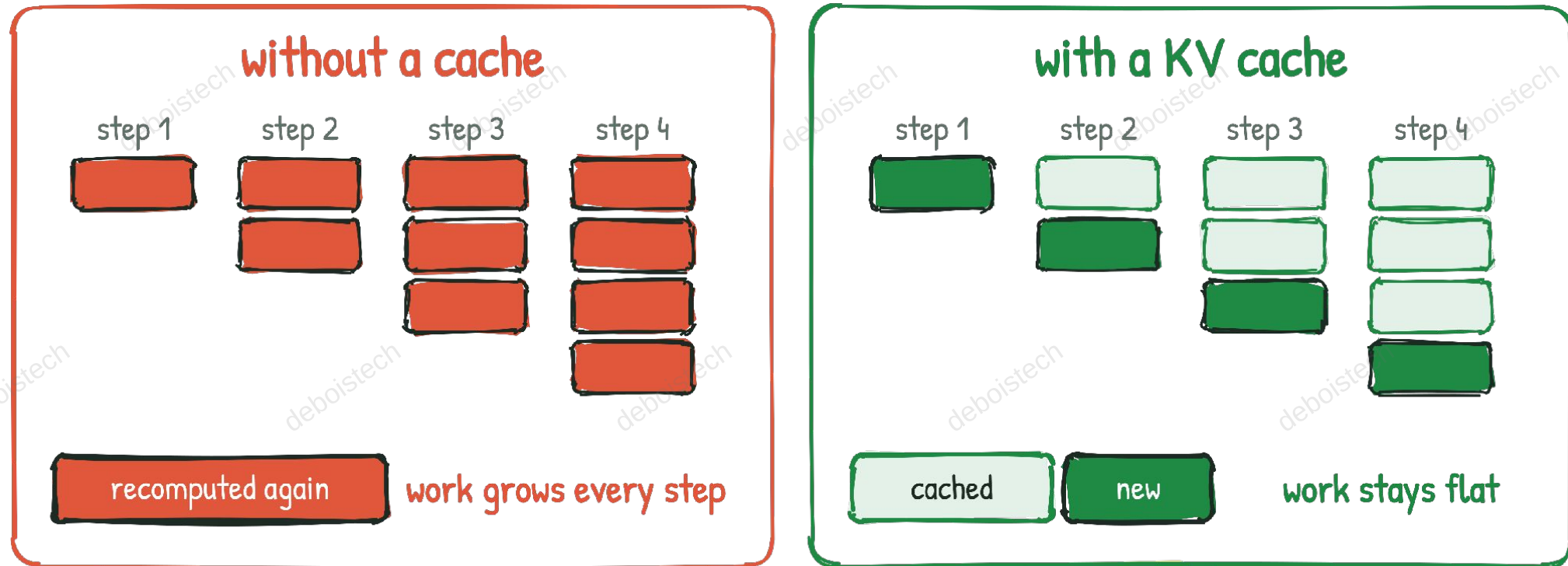
Decode: each new token depends on the tokens before it, so the model produces them one at a time. The gap between two consecutive tokens is TPOT, and it sets how fast text streams.



TPOT = time between tokens

KV cache: what it is

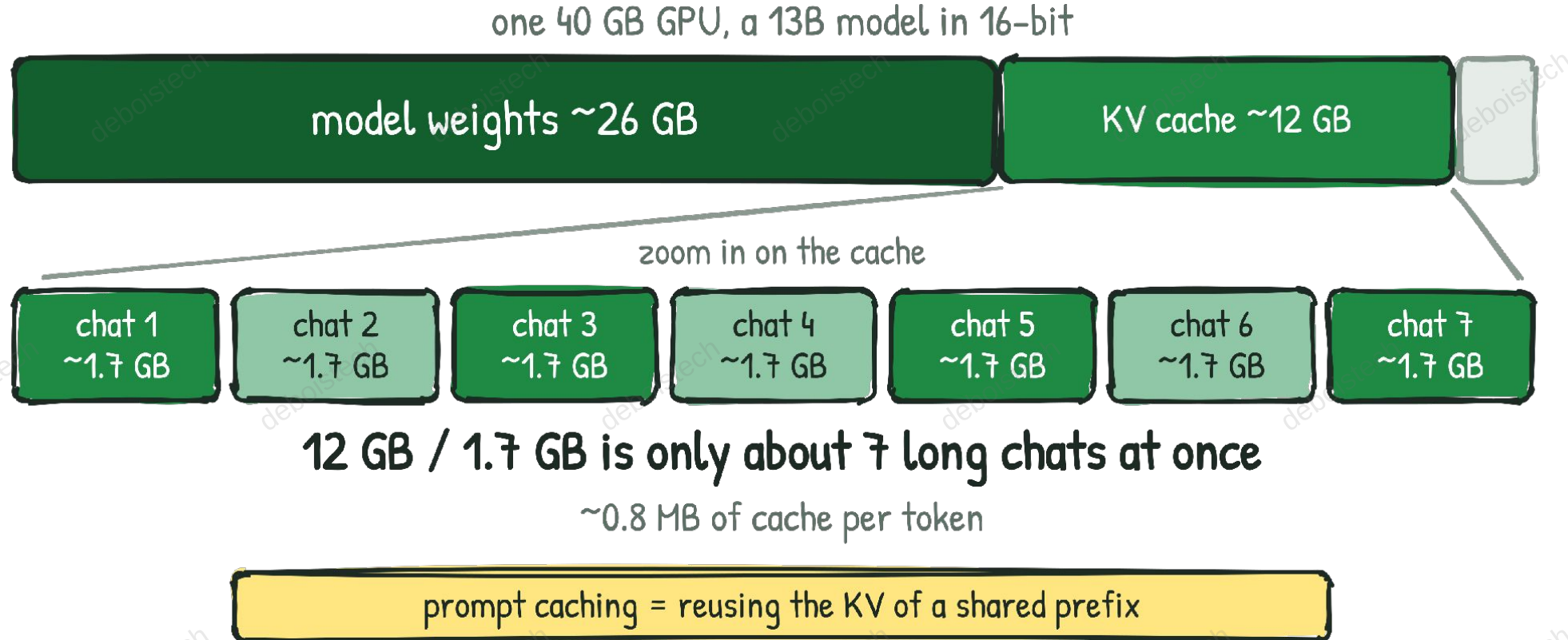
Attention needs the keys (K) and values (V) of every earlier token. Without a cache the model recomputes them for the whole sequence at every step. With a KV cache each token's K and V are computed once, stored and reused.



compute once. store. reuse.

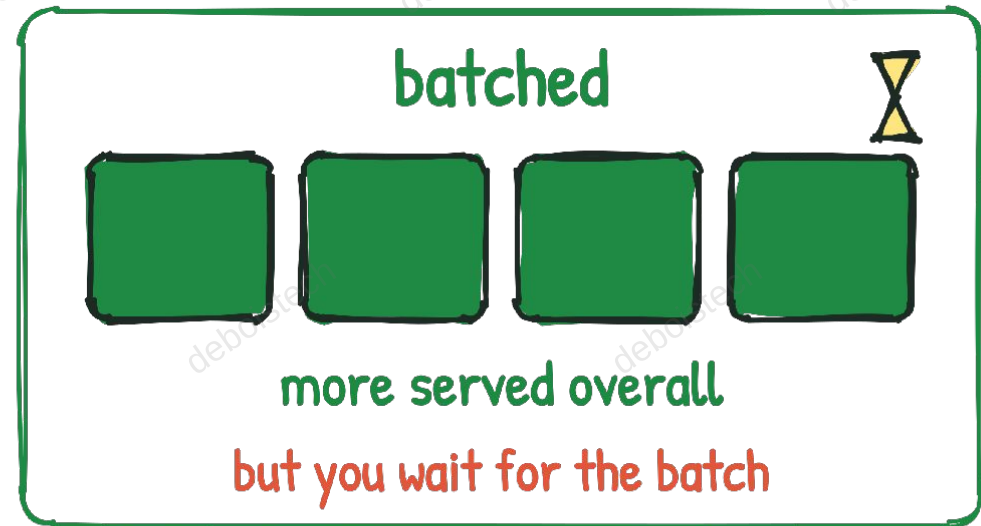
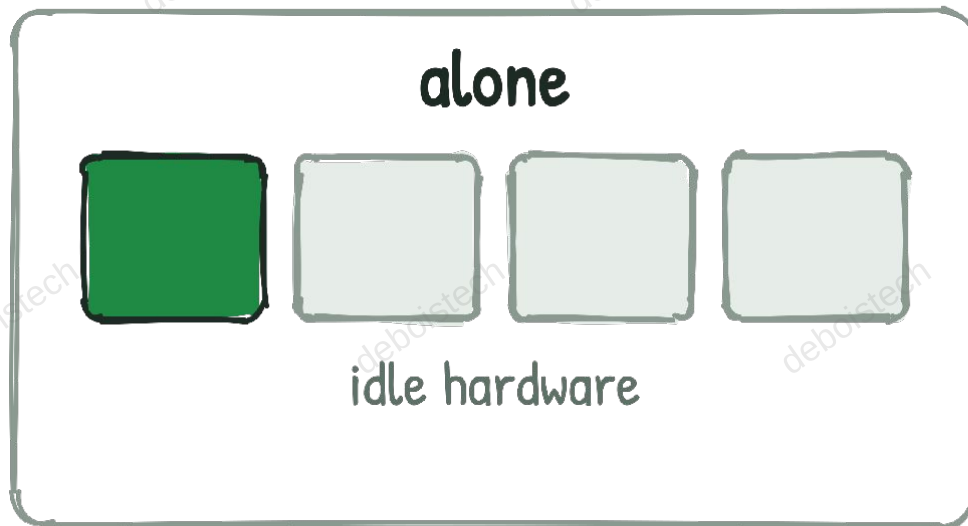
KV cache: the memory bill

The cache stores K and V for every token, in every layer, for every request in the batch: about 0.8 MB per token for a 13B model. It competes with the weights for GPU memory and sets how many requests fit at once.



Stage 5: batching

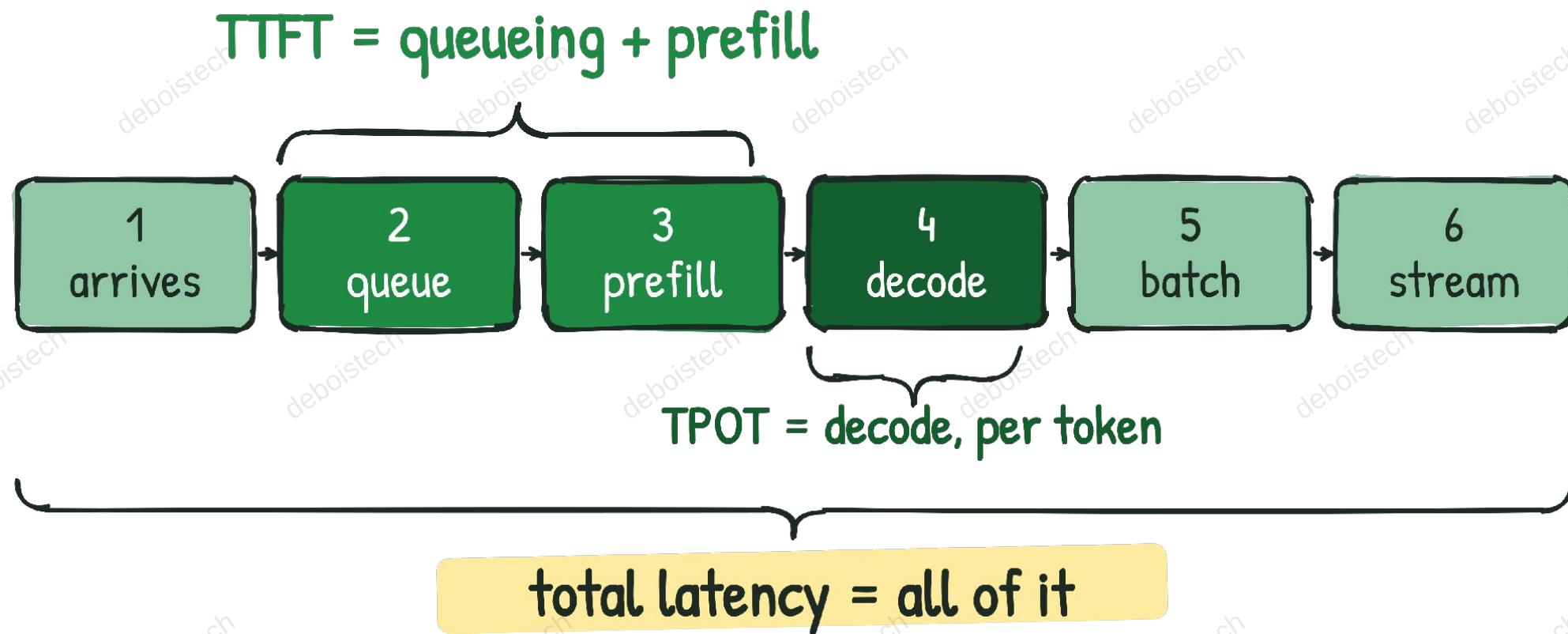
To keep an expensive GPU busy, providers batch many requests together. More requests are served per second (throughput), but your own request may have to wait for the batch (latency).



same model: fast or slow, depending on load

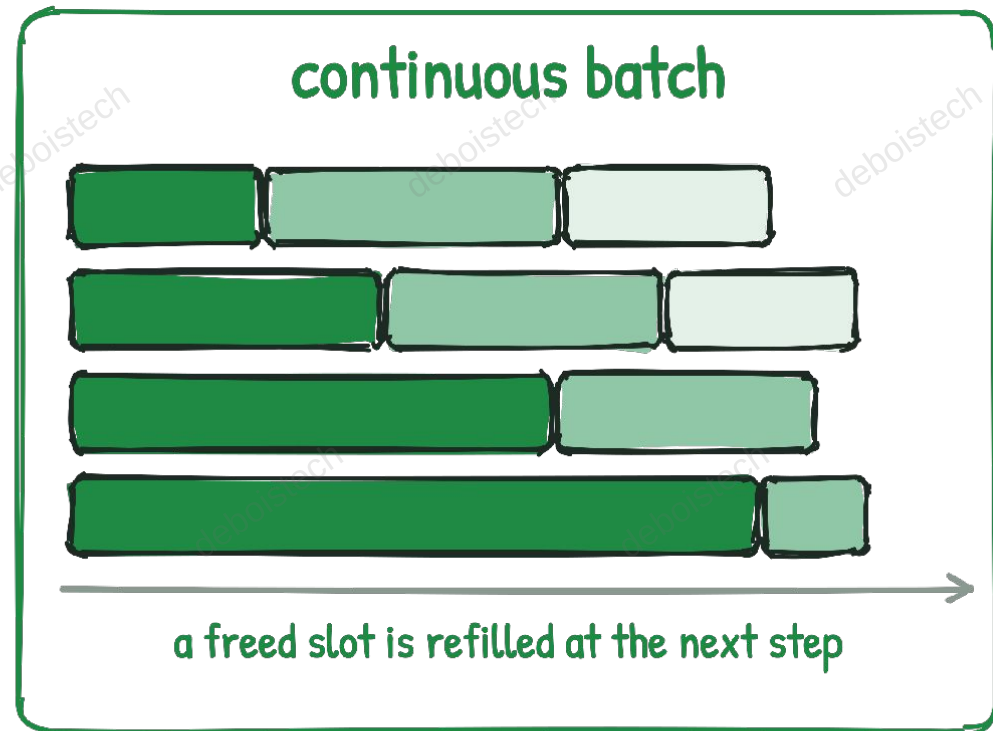
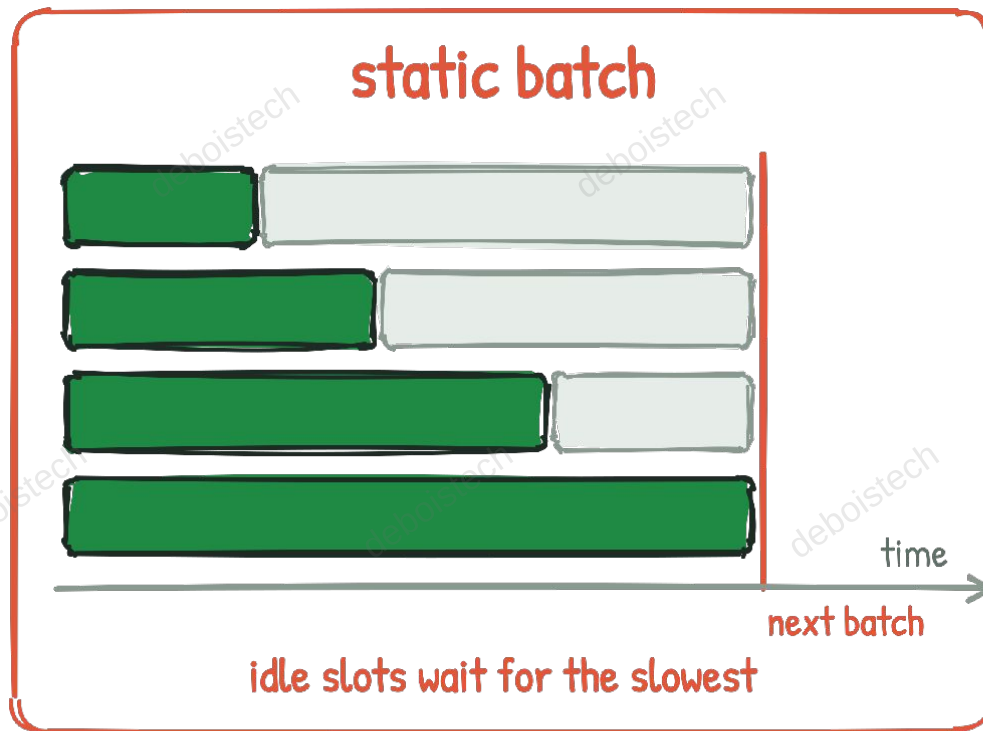
The full picture

TTFT = queueing + prefill. TPOT = decode time per token. Total latency is everything combined, roughly $TTFT + (\text{output tokens} - 1) \times TPOT$. This is the one diagram to remember from today.



Batching: static vs. continuous

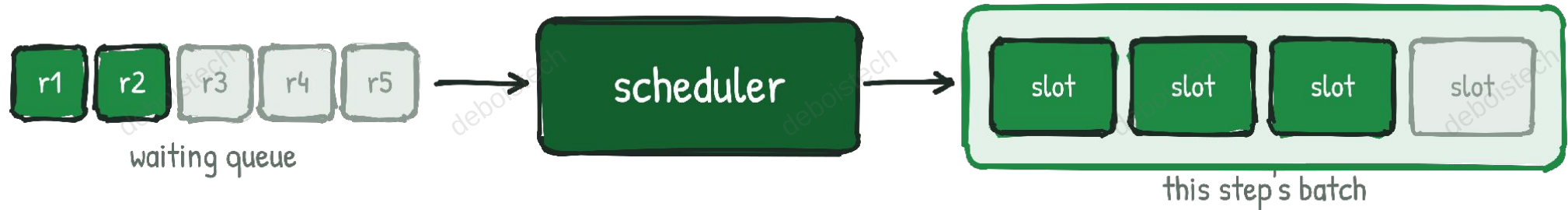
Static batching waits for the slowest request before starting anything new, so finished slots sit idle. Continuous (iteration-level) batching refills a freed slot at the very next step: the GPU stays full and newcomers wait less.



green = first requests · sage / tint = newcomers

Scheduling: who goes next?

Each step, the scheduler decides who joins the batch. A long prompt can hog a step and stall everyone's decoding, so engines slice it. When KV memory runs out they pause requests or turn them away. These choices shape your p99.



chunked prefill



green: prompt slices sage: decode

long prompts are sliced so
decoding keeps flowing

preemption



KV memory full? pause one
request, resume it later

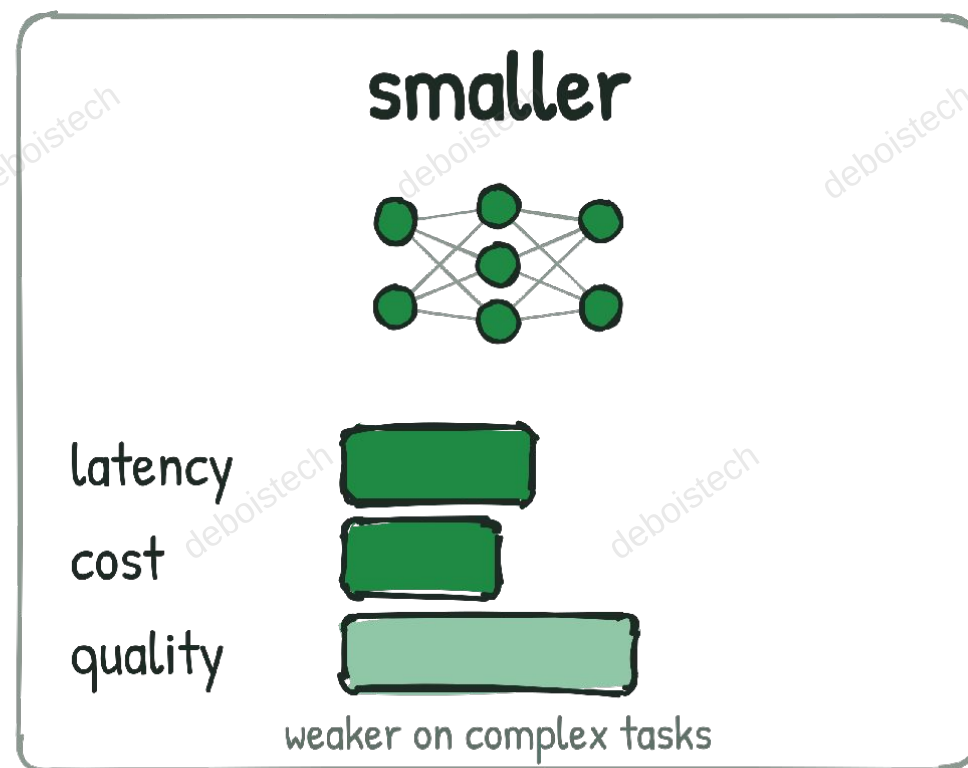
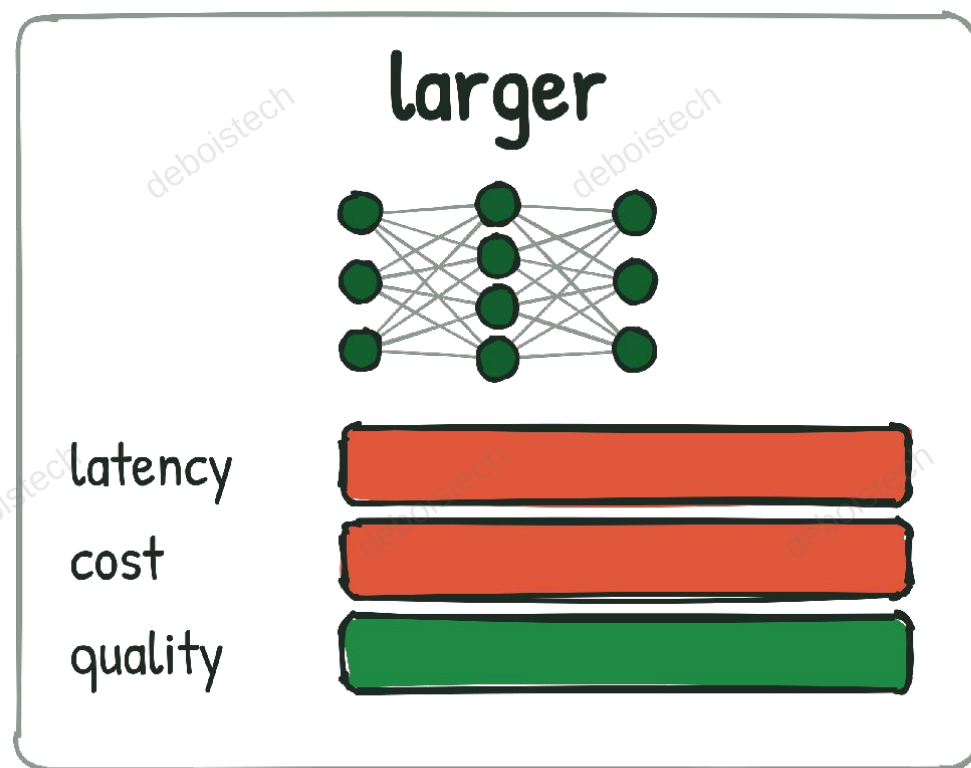
admission control



too busy? queue it,
or say no early

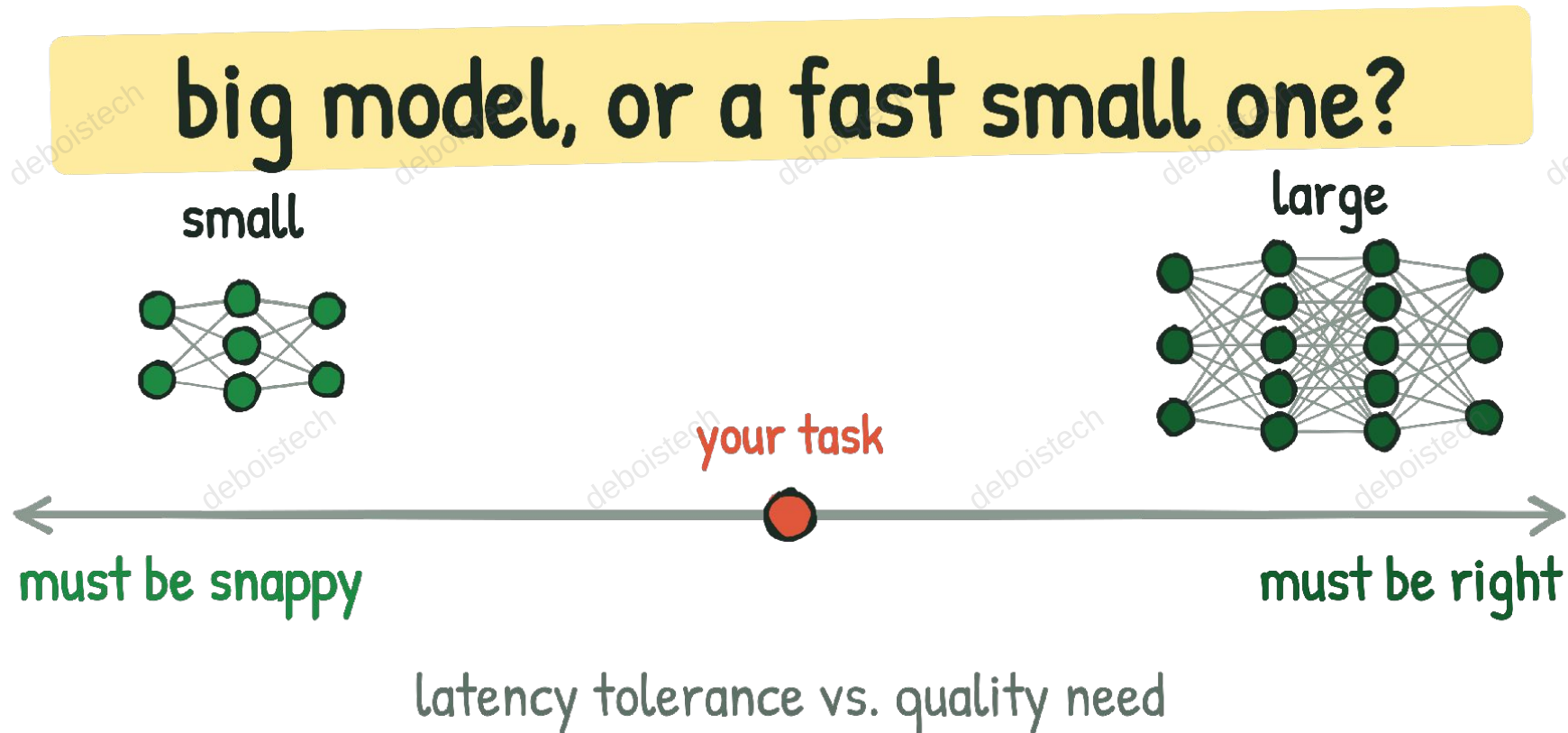
Bigger is not always better

Larger models are usually stronger but slower and pricier per token. Smaller models are cheaper and snappier but weaker on complex tasks. Latency, cost and quality move together.



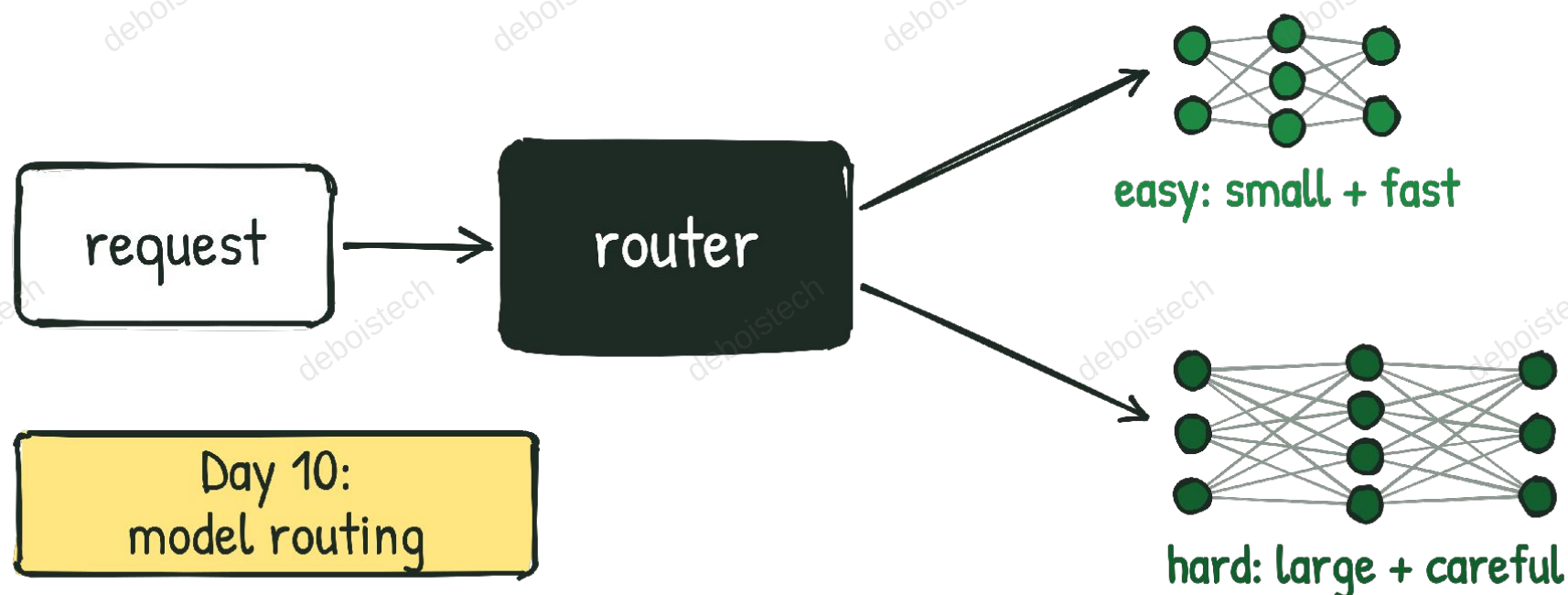
The real question

Picking a model is a trade-off, not a ranking. Ask: how long can this task wait, and how wrong can it afford to be? There is no universal answer. Which would you pick for your project? Tell us in the chat.



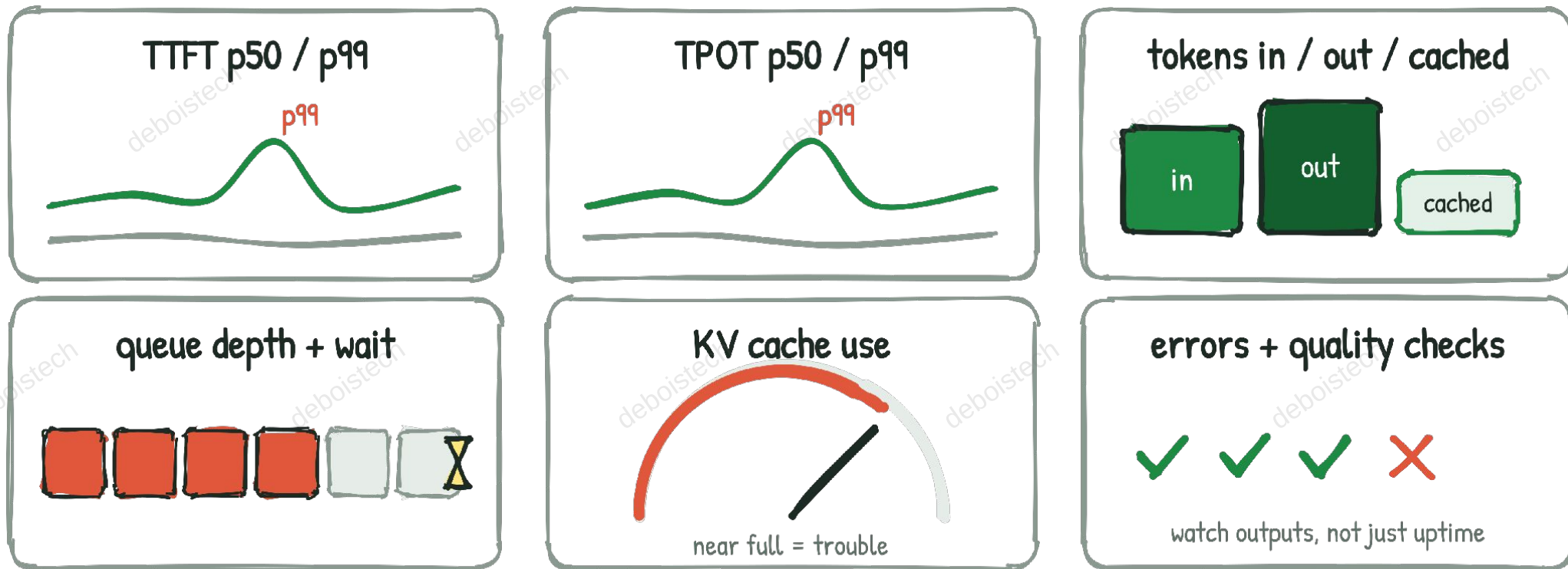
Where this is going

Instead of one model for everything, a router picks the model per request: easy ones go to a small, fast model; hard ones to a large, careful one. A soft preview of Day 10.



What to monitor

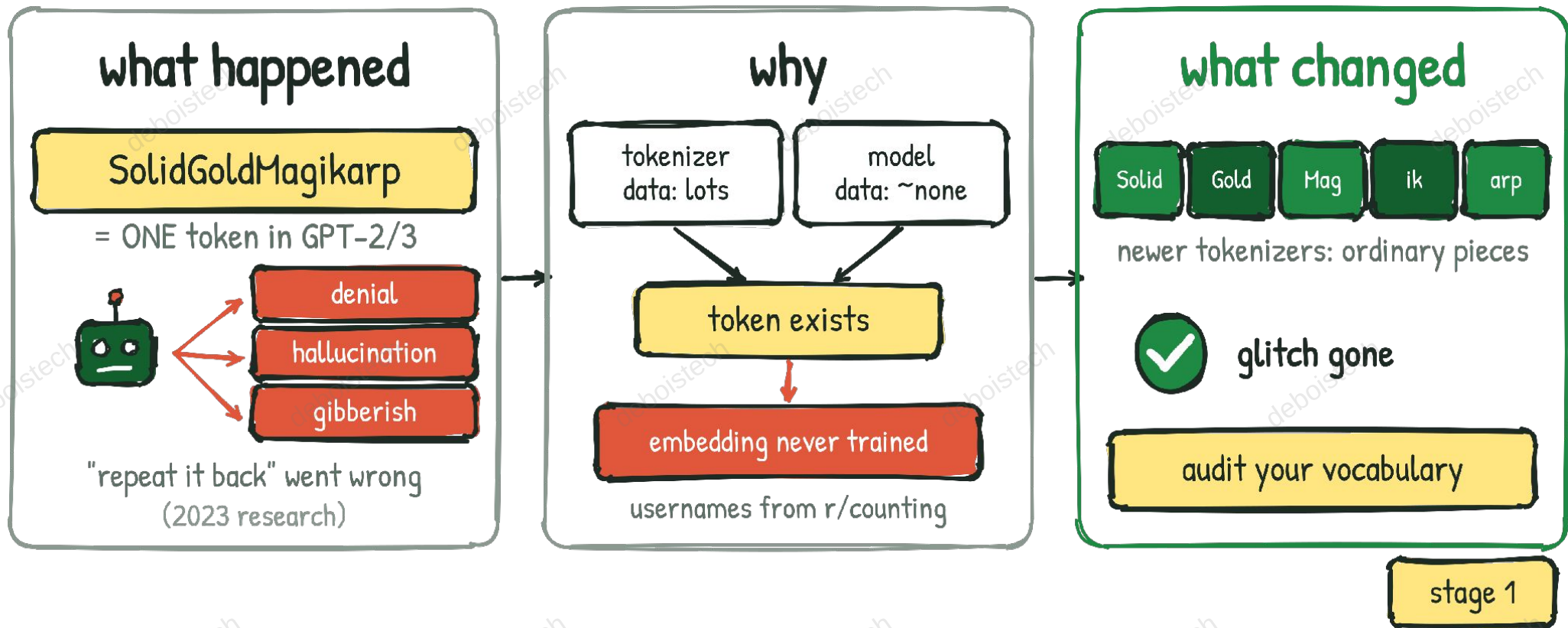
Average latency hides most of these problems. Teams watch a small dashboard: percentiles for TTFT and TPOT, token counts, queue and cache health, and output quality. Day 10 (observability) goes deeper.



Day 10: observability goes deeper

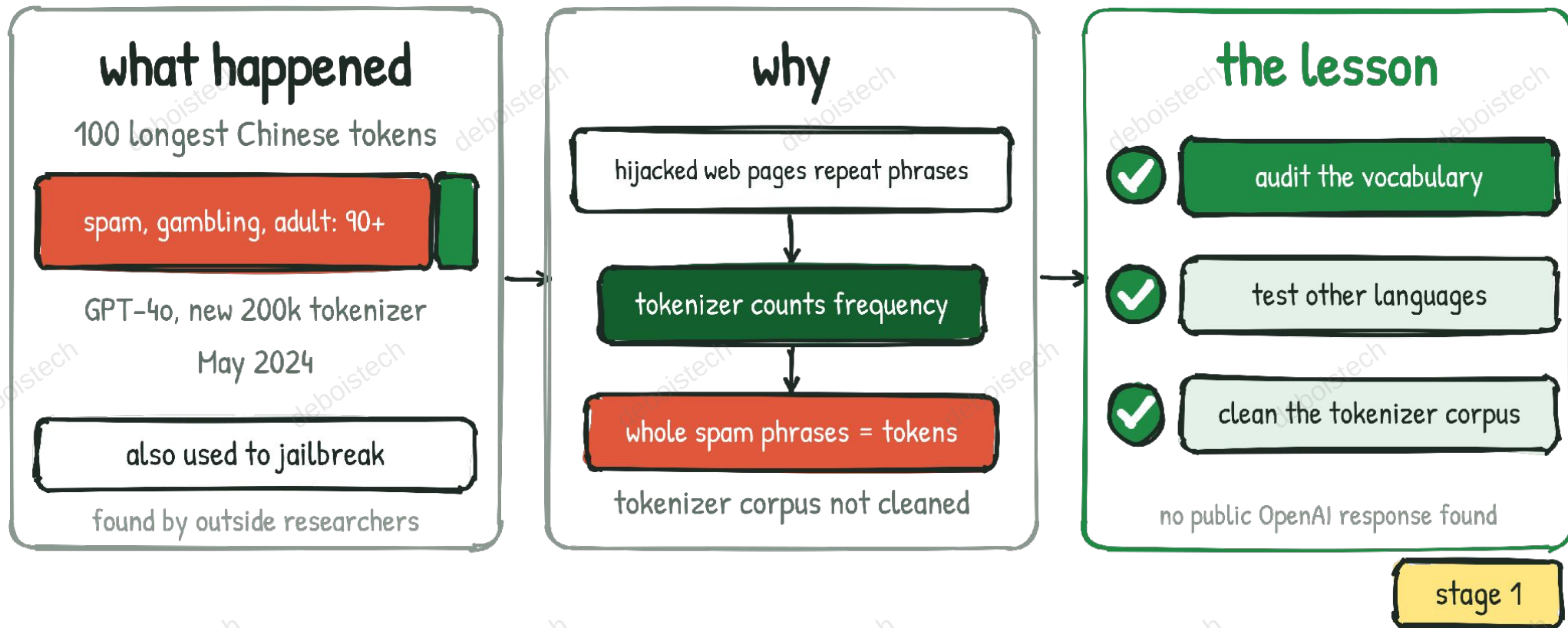
Case 1: the Reddit username token

In GPT-2/3's tokenizer, usernames from Reddit's r/counting forum such as 'SolidGoldMagikarp' became single tokens, yet barely appeared in the model's training data. Asked about them, the models denied, hallucinated or produced gibberish.



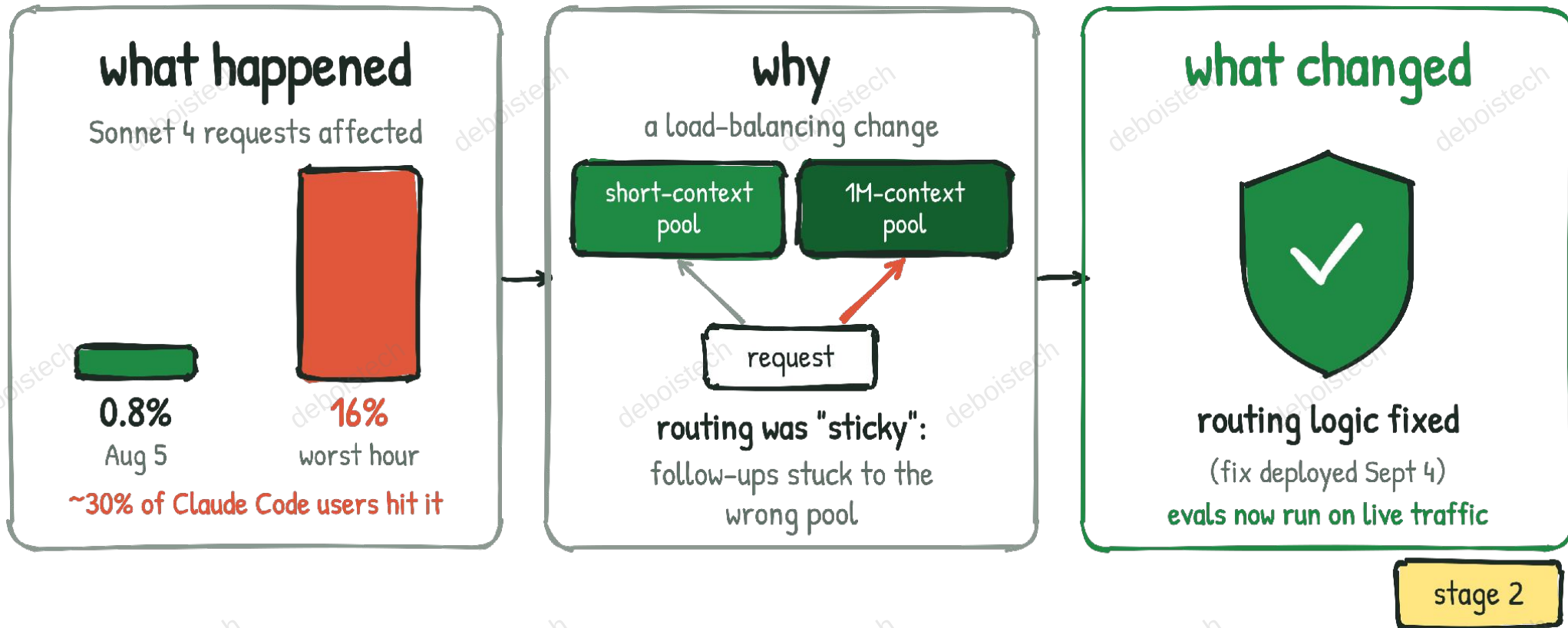
Case 2: spam in GPT-4o's tokenizer

GPT-4o launched in May 2024 with a new 200k-token tokenizer. Researchers found that most of its longest Chinese tokens were spam, gambling and adult-site phrases: the corpus the tokenizer learned from had been polluted by hijacked web pages.



Case 3: the misrouted requests

Anthropic's 2025 postmortem: a routing bug sent short-context requests to servers set up for the 1M-token context window. At the worst hour 16% of Sonnet 4 requests were affected, and sticky routing kept some users on the wrong servers.



Case 4: 60-80% of memory wasted

Berkeley's vLLM team found that 2023 serving systems wasted 60-80% of KV-cache memory on fragmentation and over-reservation, which capped batch sizes. Paging the cache (PagedAttention) cut waste below 4% and raised throughput 2-4x.

what happened

KV memory reserved per request



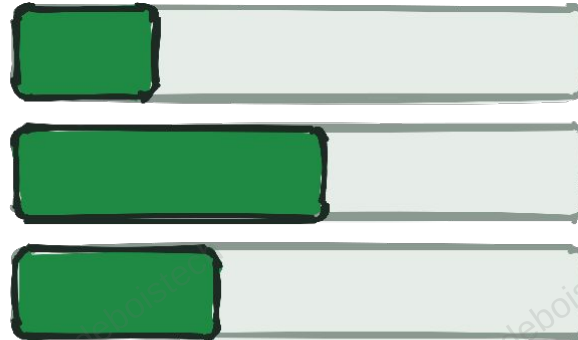
60-80% wasted

in 2023 serving systems

so fewer requests fit in a batch

why

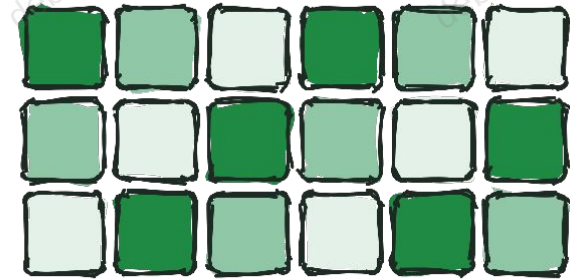
reserve for the maximum length



unused tail + gaps between slabs

the engine cannot know the
output length in advance

what changed



fixed-size blocks, any free slot

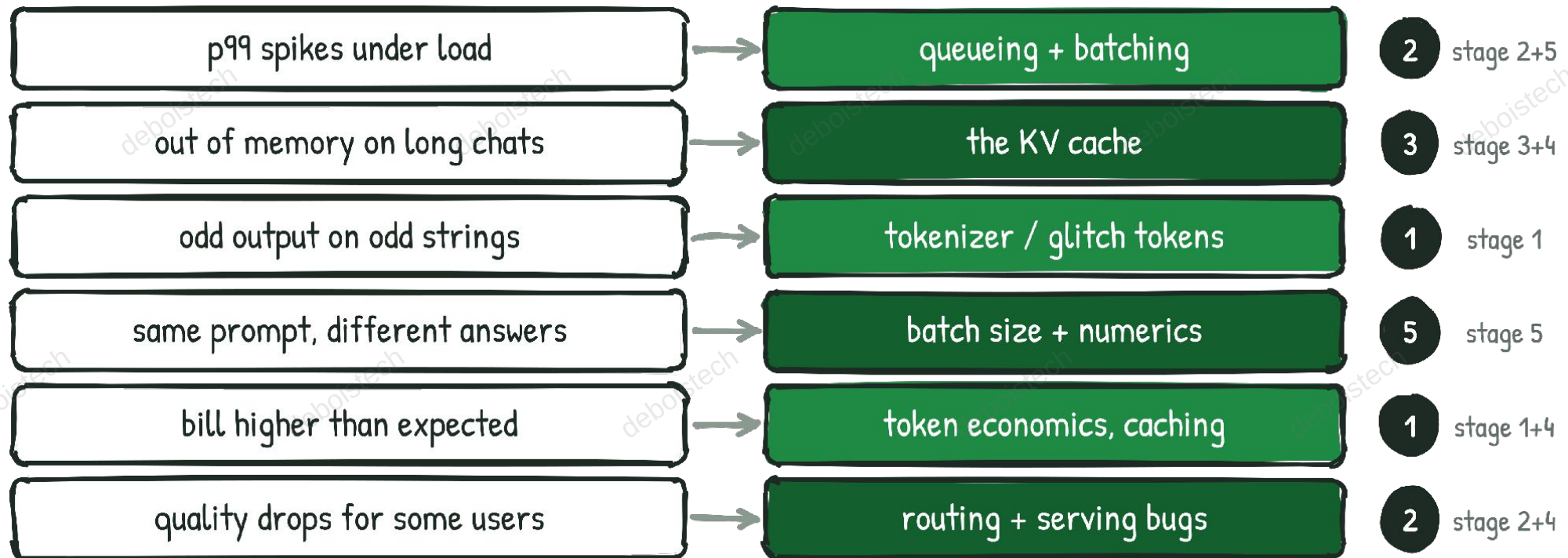
waste under 4%

2-4x throughput (paper)

stage 5

Symptom to suspect

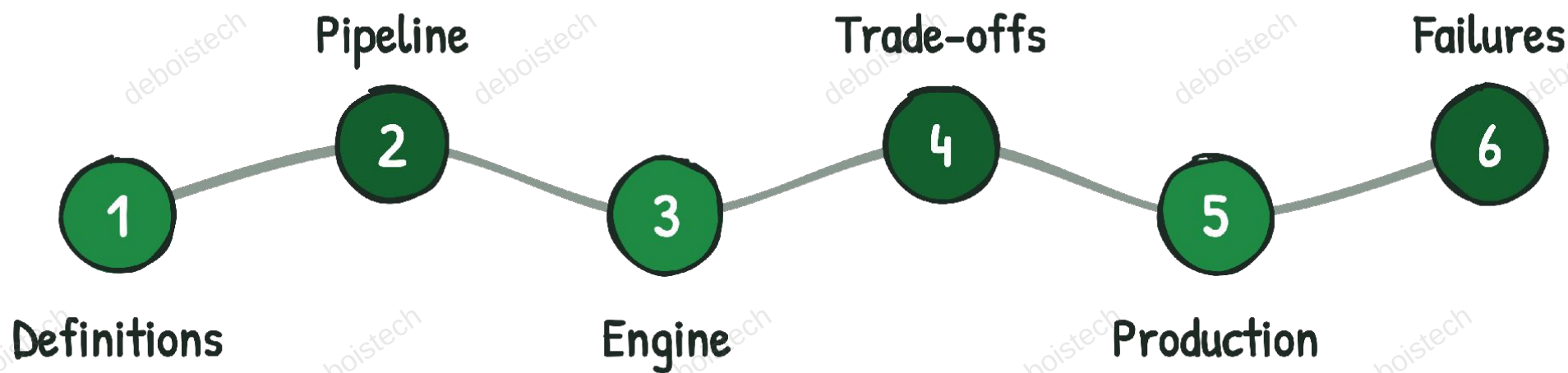
In production nobody tells you 'the KV cache is full'. You get symptoms. Knowing the concepts turns a vague complaint into a short list of suspects, and each suspect lives in one stage of the pipeline.



vague complaint in, short list of suspects out

Recap

Definitions, the six-stage pipeline, the engine underneath (KV cache, batching, scheduling, sharding), trade-offs, why it matters in production, and real failures. Remember: TTFT = queueing + prefill, TPOT = decode.



TTFT = queueing + prefill. TPOT = decode.

KV cache, batching, sharding: why it costs what it costs.

NEXT UP

Up next: Day 4

Today was about how fast an answer comes back. Tomorrow: how to make sure the answer is actually right, using retrieval-augmented generation (RAG).



Day 4

RAG

is the answer actually
right?



Sources and further reading

Every case study comes from a public write-up. Read the originals: the numbers here are as reported by their authors, and they are not independently verified.

case 1

SolidGoldMagikarp (plus,
prompt generation)

lesswrong.com

case 2

GPT-4o's Chinese
token-training data is polluted
by spam and porn websites

technologyreview.com

cases 3 + 4

A postmortem of three recent
issues

anthropic.com

case 5

Defeating Nondeterminism in
LLM Inference

thinkingmachines.ai

case 6

vLLM: Easy, Fast, and Cheap
LLM Serving with
PagedAttention

blog.vllm.ai

concepts

vLLM docs: Parallelism and
Scaling · Sarathi-Serve (OSDI
2024)

docs.vllm.ai